3 - Construire une API avec Node.js

Découvrez l'API

Qu'est-ce qu'une API ?

API est l'acronyme d'Application Programming Interface ou interface de programmation applicative ou d'application. L'API est une solution qui donne la possibilité à des applications de communiquer entre elles et de s'échanger des services ou des données. Une API permet d'accéder aux fonctionnalités ou aux données d'une application distante.

Comment fonctionne une API ?

Une requête est envoyée depuis un client vers une application ciblée. Le langage des API permet à l'application de comprendre cette requête et de réaliser les actions engendrant la livraison des contenus demandés à la machine cliente.

Développez une API

Une API permet de mettre à disposition sur le web ou sur un réseau local les fonctionnalités d'une application, ou des contenus en provenance d'une base de données ou d'un système de fichiers. L'objectif est de permettre à des systèmes d'accéder à ces fonctionnalités et à ces contenus.

Les différents types d'API

On distingue trois catégories d'API :

- L'API ouverte ou Open API disponible publiquement sans restriction.
- L'API partenaire (avec droit d'accès ou licence).
- L'API privée ou Private API développée pour les besoins interne d'une entreprise.

Les technologies des API

Il existe deux protocoles de communication sur lesquels s'appuie une API :

- SOAP pour Simple Object Access Protocol (SOAP)
- REST pour Representational State Transfer, qui s'est largement imposé. On parle d'API RESTFUL.

Pour aller plus loin : s'initier au développement d'API

Pour écrire du code pour le web, il existe un grand nombre d'API web disponibles. Vous pouvez consulter cette liste d'API.
Vous pouvez aussi vous initier aux bases du développement d'API.

Je retiens : les fondamentaux de l'API

API est l'acronyme d'Application Programming Interface ou interface de programmation d'application.
Elle peut être ouverte, partenaire ou privée.
Elle a pour objectif d'accéder aux fonctionnalités ou aux données d'une application distante.
L'objectif est de permettre à des systèmes d'accéder à ces fonctionnalités et contenus.
REST est le protocole de communication le plus utilisé pour développer une API.

Découvrez Javascript côté serveur

Les premières tentatives d'exécution de JavaScript ont lieu au milieu de la décennie 1990.

C'est Netscape Enterprise Server qui fait la première d'entre elles lors de l'introduction de JS dans son navigateur Netscape Navigator 2.0 en 1995.

En 1997, Netscape développe le projet Rhino. C'est l'un des projets liés à la refonte de Netscape Navigator en Java. Rhino est un environnement d'exécution JavaScript sous licence libre. Ce projet a permis aux développeurs de s'essayer à d'autres projets qui mettent en œuvre du Javascript côté serveur. C'est le cas de APE (Ajax Push Engine), on est au début des années 2000. JavaScript s'avérant être un bon choix pour le partage de code entre client et serveur, le projet Comet, l'ancêtre des WebSockets, l'utilise. Node.js va faire son apparition.

Découvrez Node.js

Node.js est un environnement d'exécution JavaScript multiplateforme permettant de réaliser des sites web programmés en JavaScript côté serveur. Node.js combine le moteur JavaScript Chrome V8 avec une automatisation intelligente (API d'E / S) et une boucle d'événements.

Genèse

L'idée de créer Node.js germe dans l'esprit de Ryan Dahl lorsqu'en observant la barre de progression d'un chargement de fichier sous Flickr, il constate que le navigateur doit adresser une requête au serveur web pour savoir quel pourcentage du fichier est chargé. Il décide de simplifier cette procédure et, avec son équipe, il le développe et le maintient un temps. Puis, il est parrainé par

l'entreprise Joyent. Ryan Dahl présente le projet le 8 novembre 2009 lors du premier JSConf en Europe. À ses débuts, l'environnement Node n'est portable que sous MacOS et Linux.

Principe

Le principe de Node.js est de déclarer son serveur Http et ses messages http et de mettre en place le routage des requêtes et la récupération des paramètres saisis dans l'URL. Il faut également mettre en place un système d'envoi de contenu au client en fonction de l'URL et une gestion des accès au système de fichiers, ce qui permet à l'utilisateur de lire, d'écrire ou d'envoyer un fichier. Rapidement, des frameworks comme Express.js apparaissent pour simplifier cette mise en place de base.

Pour aller plus loin : Rhino, Comet et Ajax

Voici quelques liens pour en savoir plus sur les environnements évoqués dans le cours.
La documentation de Rhino : **http://mozilla.github.io/rhino/**
Une évolution du projet Comet : **https://cometd.org/about/**
Ajax Push Engine : **https://github.com/APE-Project/APE_Server**
La documentation de Node.js : **https://nodejs.org/en/docs/**

Sous Windows

Installez le runtime Node.js et le gestionnaire de package npm.

- Accédez à Node.js
- Téléchargez la version LTS
- Exécutez le fichier **msi** pour installer Node.js (fichier téléchargé)
- Sous Windows Node vient avec les outils de la suite **Chocolatey**. Acceptez son installation quand le processus vous demandera de l'installer.

Sous Linux (Ubuntu et systèmes basés sur Debian)

- Dans le terminal, tapez la commande : **sudo apt-get install node**
- Installez le package manager npm en tapant : **sudo apt-get install npm**

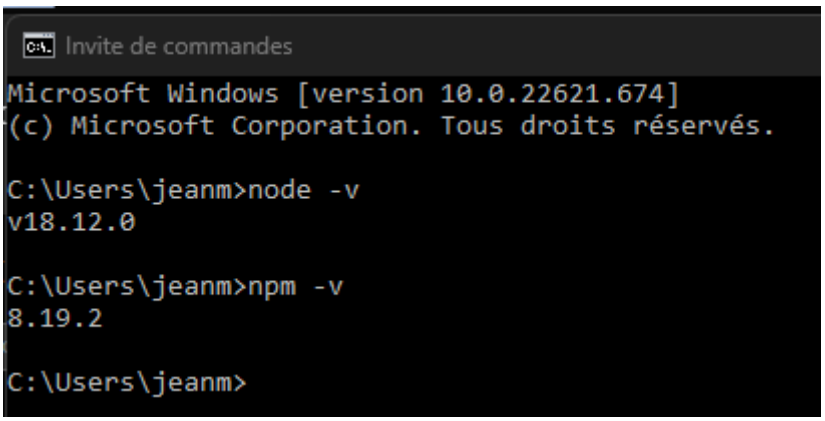
Sous MacOs

Installez le runtime Node.js et le gestionnaire de package npm.

- Accédez à Node.js
- Téléchargez la version LTS
- Exécutez le fichier **dmg** pour installer Node.js

Vérifiez l'installation

Dans le terminal, tapez :
node -v puis **npm -v**. La version installée doit s'afficher.



Gérez vos versions avec NVM

NVM est un gestionnaire de version pour Node.js. Il permet d'installer n'importe quelle version de node et de switcher entre elles.

Pour Windows

Pour installer nvm sous Windows, suivez les instructions dans le dépôt github de nvm.

Pour Linux

Dans le terminal, tapez :
sudo apt install curl
curl https://raw.githubusercontent.com/creationix/nvm/master/install.sh | bash

Puis, après l'installation :
source ~/.bashrc

Pour MacOS

Dans le terminal, tapez :
brew update
brew install nvm
mkdir ~/.nvm # Répertoire de travail de NVM
Pour le configurer :
nano ~/.bash_profile # Si vous utilisez bash
nano ~/.zprofile # Si vous utilisez ZSH
export NVM_DIR=~/.nvm
source $(brew --prefix nvm)/nvm.sh # Rend accessible la commande 'nvm' par la console
source $(brew --prefix)/Cellar/nvm/$(nvm --version)/nvm.sh # Rend accessible les commandes 'Node' par la console

Les commandes nvm

Vérifiez la version de node active :
node -v
Listez les versions de node installées :
nvm ls
Installez une version de node spécifique :
nvm install 6.9.2
Définir la version par défaut :
nvm alias default 6.9.2
Switcher de version:
nvm use 6.9.1

Je retiens : Node.js

Avant Node.js, il y a eu d'autres projets d'environnement d'exécution javascript comme Rhino et Comet.

Node.js est un environnement d'exécution JavaScript multiplateforme créé et présenté par Ryan Dahl en 2009.

Node.js utilise le moteur JavaScript ChromeV8. Son principe se base sur la déclaration d'un serveur Http, d'un système de routage, d'un système de récupération des paramètres de l'URL et un dispositif de gestion des fichiers.

Node.js est bien multiplateforme puisqu'il peut être installé sous Windows, Linux et MacOS.
Il vient avec npm qui est un gestionnaire d'extensions (package manager).
On utilise nvm pour installer toutes les versions de Node.js et switcher entre elles.

Découvrez NPM

NPM (Node Packager Manager) est le plus grand registre de logiciels du monde. Les développeurs utilisent npm pour partager des paquets. Un projet Node.js l'utilise également. Npm est constitué de trois composants : le site, l'interface et le registre.

Son site Web

Utilisez le site Web pour découvrir des paquets, configurer des profils et gérer d'autres aspects de votre expérience avec npm. Par exemple, vous pouvez configurer des organisations pour gérer l'accès aux paquets publics ou privés.
Faites le test, recherchez par exemple la librairie chart.js sur le site npmjs.

Son interface en ligne de commande

L'interface en ligne de commande, ou CLI, s'exécute avec des commandes à partir d'un terminal et constitue la manière dont la plupart des développeurs interagissent avec npm.

Son registre

Le registre est une grande base de données publique de logiciels JavaScript et des méta-informations qui les entourent. Certaines entreprises utilisent leurs propres registres privés. C’est une possibilité offerte par NPM.

Pour utiliser les paquets npm dans vos projets, il faut initialiser votre projet. À la racine de votre projet, dans le terminal, tapez :
npm init
Cette commande va créer un fichier package.json.

Installez un paquet avec NPM

Pour installer les paquets provenant d’un registre npm dans vos projets, à la racine de votre projet, dans le terminal, tapez :
npm install « nom_du_paquet »

Cette commande va créer un répertoire **node_modules** dans lequel seront installés les paquets, on appellera ces paquets les **dépendances** de votre projet. Cette commande possède des options telles que **-g** pour installer un paquet de manière globale sur votre système, ou **- - save** pour ne l'utiliser que dans un projet.

On peut aussi utiliser l’option **-dev** pour signaler au package.json que le paquet ne servira que pour la phase de développement et ne fera pas partie de l’application en production. Pour désinstaller un paquet : **npm uninstall « nom_du_paquet »**

Utilisez les registres NPM

Pour résoudre les paquets par nom et version, npm communique avec un site web de registre qui utilise la spécification **CommonJS Package Registry** pour lire les informations (c’est le cas dans le package.json).

Par défaut, npm est configuré pour utiliser le registre public npm à l'adresse https://registry.npmjs.org. Vous pouvez configurer npm pour qu'il utilise le registre compatible de votre choix, et même exécuter votre propre registre (voir les commandes **npm login**, **npm config** et **npm adduser**). L'implémentation du registre de paquets de npm prend également en charge plusieurs API d'écriture, afin de permettre la publication de paquets et la gestion des informations de compte utilisateur.

Pour aller plus loin : le package jSon

Approfondissez en consultant la documentation de NPM.
Voir package.json pour plus d'informations sur le contenu du fichier package.json.

Je retiens : NPM
Npm est un registre de logiciels composé d'un site web, d'une interface en ligne de commande et d'un registre.
Il permet d'initialiser un projet avec « npm init » et d'y installer des extensions (paquets ou modules) avec « npm install ».

Optez pour une architecture modulaire

Pour développer une application il existe deux grands types d'architectures : les **architectures monolithiques** et les **architectures modulaires**. La seconde option est conseillée et en voici la raison en détails.

L'architecture monolithique désigne un système conçu d'un seul bloc qui va contenir l'ensemble des éléments indispensables à son fonctionnement. Cette approche présente plusieurs inconvénients :

- Les applications et systèmes monolithiques sont difficiles à maintenir et à faire évoluer ;
- Il est difficile d'y intégrer de nouvelles fonctions sans remettre en cause l'ensemble de l'application.

C'est la raison pour laquelle vous utiliserez une architecture modulaire pour décomposer votre application. Avec JavaScript, cela implique de construire des applications décomposées en une suite de petits programmes/fonctions qui peuvent fonctionner à la fois indépendamment les uns des autres ou bien ensemble quand cela est nécessaire.

Identifiez les modules dans Node.js

Lorsque vous installez une dépendance dans un projet Node.js, vous faites l'action d'ajouter un **module**. Vous remarquerez d'ailleurs que ces modules sont déclarés dans un répertoire **node_modules**.
Les modules sont un ensemble de fonctionnalités, disponibles sous forme d'API, qu'un projet consommateur peut utiliser (on dit qu'un projet consomme une API).

Dans votre application, vous avez besoin d'interagir avec le système de fichiers de l'ordinateur pour lire des fichiers, les modifier en écriture ou les supprimer. Vous utiliserez alors le module **fs.**
Pour utiliser le code de ce module au niveau d'un de vos fichiers, vous déclarez au début **const fs = require('fs')** ou **import * as fs from 'fs'**. À partir de là, vous pouvez utiliser les fonctions du module comme **appenFile()** par exemple, qui permet d'ajouter du contenu à un fichier.

```
const fs = require('fs')

fs.appendFile('fichierr.txt', 'contenu à ajouter', function (err) {
    if (err) throw err
    console.log('Fichier créé !')
    }
);
```

Vous pouvez aussi l'importer de cette manière (ES6) :

```
import * as fs from 'fs';

fs.appendFile('fichierr.txt', 'contenu à ajouter', function (err) {
    if (err) throw err
    console.log('Fichier créé !')
    }
);
```

Vous pouvez également décider de n'importer que quelques-unes ou même une seule des fonctions du module, comme dans l'exemple ci-dessous où vous n'importez qu'**appenFile.**

```
import { appendFile } from 'fs';

appendFile('fichierr.txt', 'contenu à ajouter', function (err) {
    if (err) throw err
    console.log('Fichier créé !')
    }
);
```

À terme, vous pouvez également créer vos propres modules.

```
index.js > ...
1  const fs = require('fs');
2
3  fs.appendFile('test.txt', 'Contenu du fichier', (err) => {
4    if (err) {
5      console.log('Une erreur est apparue')
6    } else {
7      console.log('Le fichier a été créé')
8    }
9  })
```

Note : Création d'un fichier via la méthode « appendFile »

```
1   const fs = require('fs');
2
3   // fs.appendFile('test.txt', 'Contenu du fichier', (err) => {
4   //   if (err) {
5   //     console.log('Une erreur est apparue')
6   //   } else {
7   //     console.log('Le fichier a été créé')
8   //   }
9   // })
10
11  fs.readFile('test.txt', 'utf-8', (err, data) => {
12    if (err) {
13      console.log('Une erreur est apparue')
14    } else {
15      console.log(data)
16    }
17  })
```

Note : Lecture d'un fichier avec la méhode « readFile »

Pour aller plus loin : Node.js Modules

Approfondissez votre connaissance des modules en vous exerçant ici.

Déboguez via le module console

Tout développeur qui code côté navigateur a utilisé **console.log** lors du **débogage**. Node.js propose une console intégrée à cette fin.
Attention, nous travaillons côté serveur, il prend en charge **stdout**, **stdin** et **stderr** et pas la console du navigateur, ce qui signifie qu'à chaque fois que vous entrerez un console.log(), le contenu du log sera affiché directement dans le terminal dans lequel vous aurez lancé le serveur Node pour votre application.
Le **module console** est devenu le gestionnaire de la majeure partie de la fonctionnalité de sortie standard de Node.js.

console.log()

Exemples d'utilisation :
console.log('Hello world!')
console.log('Le nom du fichier est :', __filename)

Le premier exemple imprime la chaîne “Hello World” sur stdout.
Le second exemple montre que vous pouvez afficher le contenu de variables.

console.error()

console.error() fonctionne de la même manière que console.log, mais la sortie se fait sur stderr et pas stdout. C’est une différence importante, car stderr est toujours écrit de manière synchrone, c’est-à-dire qu’il bloquera le processus tant que la sortie n’est pas écrite.

console.dir()

console.dir() est un alias de **util.inspect()** ; il est utilisé pour énumérer les propriétés des objets.

console.time() & console.timeEnd()

Permettent de déterminer le temps nécessaire pour effectuer les actions entre les appels **console.time** et **console.timeEnd**.

```
console.time('monTimer');
var string = '';
for (var i = 0; i < 300; i++) {
    (function (i) {
        string += 'abc' + i.toString();
    })(i);
}
console.timeEnd('monTimer');
```

console.trace()

console.trace() imprime une trace de pile dans votre code sans lancer une erreur. Cela peut être utile si vous voulez savoir d'où une fonction défaillante a été appelée.

Accédez à des fonctions utilitaires avec util

Si votre code s'exécute dans Node.js, **util** est intégré. Le module util permet d'accéder à certaines fonctions utilitaires, voir la liste des fonctions d’util.
debuglog() : écrit les messages de débogage dans **l'objet error.**
deprecate() : marque la fonction spécifiée comme étant obsolète.

format() : formate la chaîne de caractères spécifiée, en utilisant les arguments spécifiés.
inherits() : hérite des méthodes d'une fonction dans une autre.
inspect() : inspecte l'objet spécifié et le retourne sous forme de chaîne de caractères.

Accédez au système de fichiers avec file

Le module fs pour FileSystem permet d'accéder au système de fichiers de la machine. On peut ainsi créer, lire, déplacer ou supprimer des fichiers.
Dans le cas d'un serveur web, c'est ce module qui est utilisé pour créer des fichiers de logs. Dans le cas d'un outil CLI, vous pourrez développer des scripts de rangement de fichiers ou d'archivage.

Pour créer un nouveau fichier, on utilise **fs.appendFile()**, **fs.open()** ou **fs.writeFile()**.
La fonction **fs.readFile()** permet de lire un fichier.
Pour écrire dans vos fichiers : **fs.appendFile()**, **fs.writeFile()**, **fs.write()**, **fs.createWriteStream.**

Découvrez les modules events & timer

Le module events

Il permet d'émettre des évènements et de les appeler dans d'autres fichiers. Voici comment émettre un événement (event) :

```
const event = require('events');

let evenement = new event.EventEmitter();


evenement.on('inscription', function(params){
    console.log(params.prenom);
    console.log(params.motDePasse);
})

module.exports = evenement;
```

Pour l'appeler :

```
evenement.emit('inscription', {prenom : 'Jim', mot
```

Le module timer

Il contient des fonctions qui exécutent du code après une période définie.
clearImmediate() : annule un objet Immédiat.
clearInterval() : annule un objet Interval.
clearTimeout() : annule un objet Timeout.
ref() : rend l'objet Timeout actif.
setImmediate() : exécute immédiatement une fonction donnée.
setInterval() : exécute une fonction donnée à chaque intervalle en millisecondes.
setTimeout() : exécute une fonction donnée après un temps donné (en millisecondes).
unref() : empêche l'objet Timeout de rester actif.

Je retiens : l'architecture modulaire de Node.js

Pour développer une application, il existe deux grands types d'architecture : les architectures monolithiques et les architectures modulaires.

Une architecture modulaire permet de découper votre application.

Les modules Node.js sont un ensemble de fonctionnalités disponibles sous forme d'API qui peuvent être importées dans vos fichiers js.

Le module console offre des fonctions de débogage sur stdIn, stdout et stderr.

Le module fs permet d'accéder au système de fichiers de la machine pour écrire, lire ou supprimer un fichier.

Le module util possède quelques méthodes utilitaires comme format().

Le module events permet d'émettre des évènements et timer d'exécuter du code après une certaine période.

Déclarez des processus fils

Les développeurs créent généralement des processus fils - on dit aussi enfants - pour exécuter des commandes sur leur système d'exploitation.
Pour la création de processus fils, il existe le module **child_process** qui offre la possibilité de générer des sous-processus. Cette capacité est principalement fournie par la fonction **child_process.spawn()**. Cette méthode génère le processus enfant de manière **asynchrone**, sans bloquer la boucle d'événements Node.js. Elle fournit les mêmes fonctionnalités de manière

synchrone. Par défaut, les canaux pour **stdin**, **stdout** et **stderr** sont établis entre le processus parent et le processus fils. On peut également générer un processus fils avec la méthode **exec().**

```
// Importation des méthodes du module
const { spawn, exec } = require('child_process');

// Déclaration d'un sous_processus (ici la commande ls du terminal)
// En second argument de la méthode on passe les options: -l pour avoir
//tout les détails de la donnée et h pour avoir un format lisible par l'humain par exemple
// En second argument on lui passe le répertoire courant du processus fils
const ls = spawn('ls', ['-lh', '/usr']);

// On exécute le processus fils quand on reçoit la donnée sur stdout
ls.stdout.on('data', (data) => {
    console.log(`stdout: ${data}`);
});
// On exécute le processus fils quand on reçoit la donnée sur stderr
ls.stderr.on('data', (data) => {
    console.error(`stderr: ${data}`);
});
// On affiche un message quand le processus fils est terminé
ls.on('close', (code) => {
    console.log(`child process exited with code ${code}`);
});

// On peut également créer un processus fils avec la méthode exec()
exec('ls -lh', (error, stdout, stderr) => {
    if (error) {
        console.error(`error: ${error.message}`);
        return;
    }
    if (stderr) {
        console.error(`stderr: ${stderr}`);
        return;
    }
    console.log(`stdout:\n${stdout}`);
});
```

Afin de vous entraîner, tentez de reproduire et de faire fonctionner le code que nous venons de voir

Créez un serveur HTTPS

Pour pouvoir écouter des requêtes entrantes et envoyer une réponse à ces requêtes, il faut au préalable créer un serveur dont le but sera justement d'intercepter et d'envoyer des requêtes.

Pour créer un serveur HTTP et commencer à écouter les requêtes, utilisez la méthode **createServer** du module **HTTPS,** le module prend en charge les certificats TLS/SSL.

La méthode **listen()** du serveur ainsi créé vous permet de définir le port d'écoute. Les paramètres **req** et **res** permettent de **manipuler les requêtes et les réponses**.

```
const https = require('https')

https.createServer(options, function (req, res) {
    res.writeHead(200)
    res.end("hello world")
}).listen(3000);
```

Créez un socket TCP ou UDP

Le socket TCP

Pour créer un socket TCP, utilisez une instance de la classe Socket du module **net**. Pour vous connecter au serveur TCP, utilisez la méthode **connect()**.
Dans le monde du développement Web et Web mobile, il sera plus rare de devoir créer un serveur TCP, mais il est toujours bon de savoir comment le faire.

```
// création du socket TCP
let localSocket = new net.Socket()
// connexion au socket
localSocket.connect(localPort, 'localhost')
// Ecriture d'un message lors de la connexion au socket
localSocket.on('connect', function () {
    localSocket.write('[local-msg]')
})
// Récupération de données et log
localSocket.on('data', data => {
    log.info(+new Date(), 'DATA_FROM_REMOTE: ', data.toString())
})
localSocket.on('error', noop)
// A la fermeture du socket
localSocket.on('close', hadError => {
    localSocket.removeAllListeners('data')
})
```

Le socket UDP

Pour créer un socket UDP, utilisez la méthode **createSocket()** du module **dgram** et la méthode **send()** du socket pour envoyer un message.
Tout comme pour le socket TCP, il sera rare de devoir créer un serveur UDP, mais il est toujours bon de voir comment faire.

Voici un exemple d'envoi de datagrammes UDP. Utilisez la méthode **send()** du socket pour envoyer un message.

```
// Importation du module
const dgram = require('dgram')
// Déclaration du socket
const socket = dgram.createSocket("udp4")

//Ici data est de la donnée en JSON
const payload = JSON.stringify(data)
//Création du message.
const message = Buffer.from(playload)
// Utilisation de la méthode send() du socket pour envoyer le message
socket.send(message, 0, message.length, 9907, '239.252.10.10', function(err, bytes) {
    if(err){
        console.log("Erreur ", err)
        socket.close()
    }
    //On peut également afficher la donnée et l'adresse du socket dans la console
    console.log(payload + socket.address().port);
});
```

Pour aller plus loin : la méthodologie des modules

Étudiez :

- Les méthodes du module child_process
- Les méthodes du module https
- Les méthodes du module net
- Les méthodes du module dgram

Je retiens : la création de processus, serveur et socket

Pour créer un processus fils ou processus enfant, on utilise la méthode spawn du module child_process.
Pour créer un serveur HTTPS, on utilise la méthode createServer() du module https.
Pour créer un socket TCP, on utilise une instance de la classe Server() du module net.
Pour créer un socket UDP, on utilise la méthode createSocket() du module dgram

Rappels : l'asynchronisme en JS

En JavaScript, le code est exécuté ligne par ligne. On est donc en mode d'exécution synchrone, car les lignes sont interprétées dans l'ordre où elles ont été écrites.

Cependant, certaines instructions ne doivent pas être exécutées immédiatement. Par exemple, si vous envoyez une requête réseau, le processus devra attendre le retour des données avant de pouvoir les traiter, occasionnant une perte de temps si aucune autre instruction ne peut être exécutée avant que la requête réseau soit terminée.

C’est dans le but de résoudre ce problème qu’on utilise la programmation asynchrone. Asynchrone signifie que vous pouvez exécuter d'autres blocs de code pendant que vous attendez la fin d'activités longues comme les requêtes réseau. Pendant la résolution de la requête réseau, le programme continue son exécution et l’utilisateur peut toujours utiliser les fonctionnalités de l’application.

Appliquez la boucle d’évènements

Quand l'interpréteur JavaScript exécute le code, chaque fonction appelée est ajoutée à la **pile d'appels,** qui est une structure de données en forme de liste où les éléments ne peuvent être ajoutés qu'au sommet et retirés du sommet. Dans une pile, si vous ajoutez deux éléments sur la pile, l'élément le plus récemment ajouté est retiré en premier.

Lorsque JavaScript rencontre une opération asynchrone, il l'ajoute à une table dans sa mémoire. Cette table stocke l'opération, la condition de son achèvement et la fonction à appeler une fois l'opération terminée. Lorsque l'opération est terminée, JavaScript ajoute la fonction associée à la **file d'attente des messages,** qui est une autre structure de données de type « liste » dans laquelle les éléments ne peuvent être ajoutés qu'au bas de la liste et retirés du haut.

La **boucle d'événements** est un processus perpétuel qui vérifie si la pile d'appels est vide et y déplace les fonctions de la file d'attente des messages, en leur donnant la priorité sur les appels de fonction qu'il interprète dans le code.

Programmez en asynchrone avec les callbacks

```
const request = require('request');

request('https://pleindefilm.com/films', (error, response, body) => {
    if (error) {
        console.error(`Impossible d'envoyer une requête à l'API: ${error.message}`)
        return;
    }

    if (response.statusCode != 200) {
        console.error(`Code d'état 200 attendu mais le code reçu est ${response.statusCode
        return;
    }

    movies = JSON.parse(body);
    movies.forEach(movie => {
        console.log(`${movie['title']}, ${movie['release_date']}`);
    });
});
```

Un **callback** (fonction de rappel) est une fonction qui est passée comme argument à une autre fonction, puis exécutée lorsque cette autre fonction est terminée. Imaginons, par exemple, que nous souhaitions envoyer une requête à une API pour obtenir une liste de films. On utilisera pour cela la méthode **request()** du module **request** de Node.Js.

Dans notre exemple, le callback qui sera exécuté lorsque la requête est terminée est une fonction fléchée qui prend ici **error**, **response** et **body** en argument. Dans le body, on trouvera la liste des films demandés.
Longtemps, les callbacks étaient le mécanisme le plus courant pour écrire du code asynchrone, mais ils sont aujourd'hui largement obsolètes car ils peuvent rendre le code confus à lire (souvenez-vous du callback hell).

Programmez en asynchrone avec les promesses

Une promesse est un objet JavaScript qui renverra une valeur dans le futur. Les fonctions asynchrones peuvent renvoyer des objets de type promesse au lieu de valeurs concrètes. La promesse a trois états :

- On obtient une valeur dans le futur, la promesse est remplie.
- On obtient une erreur dans le futur, la promesse est rejetée.
- Sinon, la promesse est en attente et toujours en cours de traitement.

Généralement, une promesse prend la forme :

```
promiseFunction()
    .then([ 'Fonction de rappel pour une promesse remplie' ])
    .catch([ 'Fonction de rappel pour une erreur' ])
```

Ce sont des fonctions précédées par le mot-clé **async**, et qui peuvent contenir le mot-clé **await**. Ces mots permettent un comportement asynchrone utilisant les promesses et évite de configurer explicitement les chaînes de promesse.

Deux façons de coder en asynchrone

```
// Voilà un exemple d'écriture sous forme de promesse
function getProcessedData(url) {
    return downloadData(url)
        .catch(e => {
        // renvoie une promesse erreur
        return downloadFallbackData(url)
        })
        .then(v => {
        // renvoie une promesse en cas de succés du téléchargement
        return processDataInWorker(v)
        });
}

// Voilà le même exemple sous forme de fonction asynchrone
async function getProcessedData(url) {
    let v;
    try {
        v = await downloadData(url);
    } catch(e) {
        v = await downloadFallbackData(url);
    }
    return processDataInWorker(v);
}
```

Pour aller plus loin : La librairie RxJs

Faites un tour du côté des promesses et de la librairie RxJs.

Je retiens : l'asynchronisme sous Node.js

Programmer en asynchrone signifie que vous pouvez exécuter d'autres blocs de code pendant que vous attendez la fin d'activités longues.

Pour coder en asynchrone, on utilise la syntaxe des promesses promise().then().catch() ou des fonctions asynchrones avec les mots clés async et await.